

FleetMesh product audit

Verdict: a credible internal-round prototype, with national-winning readiness still unproven.

FleetMesh now has meaningful backend behavior, repeatable demonstrations and inspectable results. It is suitable to present as a working software prototype for PS 26123. It should not be described as a finished industrial AMR product or a guaranteed winner.

Audit scope: FleetMesh 1.2, 12 September 2026. Reviewed frontend source and workflows, server lifecycle, persistence, local API controls, robot messaging and coordination. Fixed concrete defects and ran release checks. The available browser rejected the local address, so no full browser visual or responsive-layout pass is claimed.

Area	Assessment	Reason
Frontend	Improved; browser sign-off pending	Compact demonstration timeline, mission filters, stable controls, clearer errors and explicit process-stop confirmation.
Backend	Strong for a local prototype	Separate robot processes, direct authenticated UDP, SQLite records, input checks, retry protection and corrected lifecycle behavior.
Core functionality	Working within simulation limits	Allocation, ownership transfer, route permission, aisle handling, disruption replay and exports are exercised by tests.
Edge AI	Main technical weakness	The learned delay model is available, but the fixed rule is faster in four of five benchmark averages. Edge-board deployment is untested.
Real-world impact	Plausible; not validated	AMR integrators are a reasonable pilot audience. No measured customer ROI, adoption or physical warehouse deployment exists yet.

What is genuinely stronger now

The application can demonstrate a complete operator workflow and preserve evidence of it. The new audit adds 23 passing system tests, an expanded HTTP/UI harness and six process-based workload cases. Those six cases completed 81 jobs without detected collisions or duplicate active ownership.

What will matter more than additional styling

The supplied statement asks for edge execution, at least three AMRs, distributed conflict handling and a minimum 20% task-time reduction against stop-and-wait. The current product demonstrates the software mechanisms, but does not establish the general performance target or physical deployment. Closing those proof gaps is the highest-value next work.

Frontend and backend defects fixed

Finding in v1.1	Change in v1.2	Verification
Paused actions could be missing from reports. Reproduced: two live jobs, one saved job.	Persist accepted run actions immediately; capture current state when exporting a live run.	Regression test checks the saved mission ledger and network state.
A lost command response could lead to a duplicate user submission.	Request IDs deduplicate successful commands; the UI retries a transport failure once using the same ID.	Injected response loss after a server commit; one mission created across two requests.
Failed/timed-out runs still accepted mutations; completed runs exposed irrelevant fault controls.	Reject ended-run mutations and align UI control states. Completed runs can accept a new mission.	Terminal-state regression and UI checks.
Failed replacement startup could leave an old run identity beside an empty live state.	Clear live identity and display a startup error while retaining previous records.	Injected startup failure and checked preserved evidence.
Old records could still say running after server restart.	Mark old running/paused records interrupted. They remain replayable inspection records.	Reopen test checks status and non-playing snapshot.
Predictive mode could silently use a fallback when the model was absent or invalid.	Reject predictive mode unless three finite weights are available. Fixed routing remains available.	Invalid-model test confirms current run is preserved.
Robot cards were replaced on every poll, removing action nodes and focus.	Create action nodes once per fleet/run; update changing text and state in place.	UI harness checks action-node identity across updates.
Dense demo copy, limited mission lookup, easy accidental process termination.	Compact five-stage timeline, mission search/status filters, process details, stop confirmation and queued-closure labels.	Workflow and DOM-reference checks; visual review remains pending.

Also fixed: an old launcher version string, unknown export routes returning 200, mutable snapshot references, missing response lengths and unbounded frontend waits. Form errors now appear inside the relevant dialog, and controls disable when the local server is unavailable.

Retry protection covers the latest 256 successful command IDs within one server process. It is not a durable exactly-once guarantee across restarts or eviction. The application remains local-only and single-operator.

What the release checks establish

23 system tests passed. 25 HTTP/UI harness checks passed. All six additional process cases passed. The harness executes the application JavaScript with minimal DOM/canvas stubs against a real local HTTP/UDP backend. It does not replace Chrome, Edge or mobile-browser testing.

Map / scenario	Robots	Jobs	Time (s)	Result
compact / normal	3	9	145.7	Completed
warehouse / network	4	9	192.9	Completed
extended / network	6	12	287.0	Completed
warehouse / blocked	3	9	181.6	Completed
extended / congestion	6	12	200.0	Completed
warehouse / normal	3	30	673.0	Completed

The six configurations were declared before execution (seeds 9410-9415). Each checks completion, distinct processes, exclusive held zones, unique active job ownership, one delivery per job, no dead peers and zero detected collisions. Total: 81 jobs. These are functional checks on known map families, not a new speed comparison or an independent validation campaign.

The original benchmark is unchanged

The 160-run benchmark uses five cohorts, eight paired seeds and four policies. Predictive-policy reductions against the included sequential baseline are 11.3%, 17.7%, 24.3%, 26.1% and 18.9%. Only two cohort means exceed 20%. The fixed rule is faster than predictive routing in four of five means.

That baseline is deliberately simple and conservative: it acquires route zones sequentially and retains them to the dock. Comparing predictive mode with it combines coordination and routing improvements. It does not isolate the AI contribution or prove superiority over a commercial fleet product.

Limits on every reported result

Times are simulated seconds through final delivery and return to dock, not wall-clock warehouse measurements. Positions are exact; motion is simplified; battery is illustrative. Network faults are injected on one host. Zero detected collisions in finite tests is not a universal safety proof. No edge-board, physical-AMR, real-Wi-Fi or human-interaction trial was executed.

Inspect or reproduce

Run: `python -m unittest discover -s tests -v`

Run: `python tools/audit_runtime.py`

Optional UI harness: `node tests/ui_smoke.cjs`

Evidence: `tests.txt`, `runtime_audit.json`, `ui_http_checks.json` and `frontend_audit.json` in `evidence/`. The benchmark and recorded demonstration are separate files.

Gaps that determine competition readiness

Priority	Gap	Required evidence / decision
1	Edge and physical execution	Run controllers on three separate edge computers. Then integrate three AMRs in a supervised area with localization, braking, watchdog and stop validation.
1	Performance target	Freeze an independent workload and stronger stop-and-wait comparator. Report all completed, timed-out and failed runs, paired differences and uncertainty. Separate delivery-only and dock-return metrics.
1	AI usefulness	Compare predictive routing directly with the fixed rule on unseen workloads. Keep the fixed default until the learned model produces a consistent benefit.
1	Failed-peer progress	A failed robot can retain road ownership and halt progress. Recovery must establish physical clearance and safe membership replacement. Do not expire ownership merely on a timer.
2	Browser and device confidence	Check the actual presentation laptop and browser, keyboard navigation, laptop widths and a narrow viewport. Windows/macOS launchers have not been executed here.
2	Customer value and differentiation	Interview an AMR integrator and a warehouse operator. Validate the value of inspectable local coordination and incident replay using their workload and integration constraints.

Why I would keep this solution direction

The problem fit is real: coordination, ownership, congestion and fault behavior belong at the center of this statement. The code is small enough for the team to understand and demonstrate on laptops, and it produces inspectable records. Replacing it with an untested RL planner or an unrelated AI assistant before 16 September would not address the main evidence gaps.

What should remain outside the current claim

Do not claim novel invention of mutual exclusion, guaranteed deadlock freedom, physical safety certification, public-cloud independence of every subsystem in a deployed fleet, production cybersecurity or measured customer savings. Current authentication and local host restrictions support a local demonstration; they are not a production device-identity system or an independent security audit.

Product positioning

Present FleetMesh as a local coordination prototype and evaluation workspace for small AMR fleets. The next milestone is a partner-defined three-AMR validation pilot. Integration and ongoing support are possible business models, subject to customer validation.

Team acceptance checklist before 16 September

These checks remain for the actual presentation laptop because this environment cannot certify its browser, display or operating system. Use the included launchers; Python 3.10 or newer is required. No third-party Python package, paid API or cloud account is required by the application.

Check	Pass condition
Launch and readability	Extract the full ZIP and launch. Overview, map, mission table, dialogs and buttons are readable at your normal laptop resolution and browser zoom.
Seven-job demonstration	Launch demo. Observe all five stages and the actual completion state. Open the run report and replay; confirm the ledger matches the displayed run.
Operator controls	Filter an urgent mission; pause; add a mission; export the report. Check that the new mission is present. Close and reopen an aisle and inspect its label.
Keyboard and dialogs	Use Tab, Enter and Escape. Focus stays on the robot action while data updates. Form errors appear inside the dialog. Cancel a process-stop dialog without stopping the robot.
Failure explanation	In a disposable experiment, confirm Stop process. Explain why its road ownership remains held. Start a new experiment to restore the simulated fleet.
Submission and backup	Fill in registered team details, export the six-slide pitch to PDF, and keep the ZIP, PDF, run report and a labelled backup recording on the laptop and a spare drive.

What to say to judges

“FleetMesh is a working simulation with independent robot controllers. We can show job assignment, route permissions, blocked-aisle handling and message disruption, then inspect the recorded outcome. Our current fixed rule beats our learned model in most benchmark averages, so we keep AI optional. We seek the next round to validate the architecture on three edge-controlled AMRs and an independently defined workload.”

Sources and assessment basis

Requirements: team-supplied screenshots of SIH PS 26123 and SIH2026-IDEA-Presentation-Format.pptx. Product findings: inspected release source, reproduced API mismatch and bundled test evidence. Winner/commercial-product context: the cited FleetMesh_SIH_Winner_Research.pdf included in docs/.

Official evaluation context: search-indexed SIH 2026 college guidance mentions novelty, complexity, clarity, feasibility and practicality. The full PDF request returned 403, so this audit does not assert a complete official scoring rubric or weights. [Open SIH 2026 guidelines](#).

Assessment is engineering judgment, not a prediction of selection. Competition outcomes depend on other entries, the team’s explanation, judge assessment and the evidence presented.